

CS 510 Project Report: Trailhead

Sam Hu

University of Illinois at Urbana-Champaign
samhu2@illinois.edu

1 Artifact Link

Repo link: <https://github.com/reg223/trailhead>

Also published on the chrome extension store! link

2 Tool Description: Motivation

The modern research process is characterized by an overwhelming volume of digital information, leading to the phenomenon of "tab fatigue." While the internet provides near-infinite access to data, the tools used to navigate it—standard browsers and search engines—often fail to maintain the associative context of a deep dive. When a researcher explores a complex topic, they "blaze a trail" through disparate sources, but this mental map is lost as soon as tabs are closed or bookmarks become a cluttered list. Existing tools like Zotero are excellent for keeping an archive of cited works but do not offer substantial support for earlier phases of research, when citation alone does not provide enough information.

Trailhead is motivated by the need to bridge the gap between passive citation management and active discovery. Instead of recording a link or the full page, Trailhead records *snippets* of information that the user extracted from sources, saving the most important information from the user's perspective.

By implementing a local associative search index and visual structural views, Trailhead aims to:

- **Reduce Cognitive Load:** Allowing users to offload the "state" of their research to a digital secondary memory.
- **Combat Information Fragmentation:** Unifying snippets, full pages, and personal notes into a single, cohesive topic session.
- **Enable Nonlinear Navigation:** Moving away from chronological history toward a "mental map" that reflects how the user actually thinks and connects ideas.

On top of this generic use case, Trailhead also aims to provide better support to the STEM community, where $\LaTeX$ is commonly used.

3 Tool Description: Intended Users

Trailhead is designed for power users of the web who engage in high-intensity information synthesis and require a structured environment to manage non-linear discovery. The tool is engineered for individuals who find that traditional

browser organization—such as linear history and static bookmarks—fails to capture the complex mental state required for large-scale research projects.

The primary user groups include:

- **Academic Researchers and Students:** Individuals conducting literature reviews or multi-stage projects across disparate sources. These users benefit from "topic-based sessions" that preserve the logical progression of their research over long periods, preventing context loss.
- **STEM Professionals and Educators:** Trailhead also facilitates workflow when there is a need for recording complex math notations or BibTeX citations.
- **Information Workers and Analysts:** Professionals such as market researchers or technical analysts who must compile "dossiers" from fragmented data points. The local associative search index allows these users to retrieve specific information using conceptual relevance instead of manual navigation.

4 Tool Description: Features

4.1 select to add snippets

The primary feature of Trailhead grants the user the ability to select arbitrary chunks of text and record them. This is intentionally designed to be a lightweight operation that can be done at any moment with minimal effort.

From the tool interface, users will also be able to add their own notes to each snippet.

4.2 screenshots

While text can capture the majority of the information found, there still exists some that cannot be efficiently expressed or converted into text. Trailhead therefore also allows user to store image or screenshots for complex information such as graphs and tables.

4.3 Automatic citation & one click export

Aiming to provide a seamless experience for researchers, Trailhead also attempts to automatically fetch the DOI number when recording a snippet, if the viewed page contains such information. When the search is concluded, users can also export all citations, in BibTeX format, with a single click. Users can also manually add citations by entering DOI number or raw BibTeX.

4.4 search

As part of the effort to provide better navigation, Trailhead also implemented its own within-session search for users to fetch snippets based on the information of snippets itself and user-added notes.

4.5 graph view

Also intended to provide better navigation, the graph view for Trailhead provides user a venue to view recorded snippets in relation to each other, Highlighting subtopics via visual representations.

5 Tool Description: Implementation

The following section will attempt to introduce the specific implementation Trailhead. I will begin with an overview of the design and tech stack and then go over any noteworthy details for each compartment.

5.1 Overview

Trailhead is implemented as a browser extension to ensure seamless integration with the user's existing research workflow. By utilizing the Chrome SidePanel API, the tool provides a persistent, non-intrusive interface that can be toggled with a single click. From an engineering perspective, modern browser extensions are well-suited for the computational and storage requirements of this project; for comparison, the resource overhead of Trailhead is significantly lower than that of a standard video streaming tab.

The technical stack was selected to prioritize performance, type safety, and local-first data persistence:

- **Vite with CRXJS:** Used as the development framework to enable modern build tooling and Hot Module Replacement (HMR) specifically for extension manifests.
- **Svelte:** Employed for the front-end due to its reactive compiler-led approach, which ensures a lightweight and highly responsive UI.
- **IndexedDB + idb:** Acts as the local-first database, providing a robust, asynchronous storage solution for large quantities of research snippets and associated metadata.
- **TypeScript:** Utilized across the entire stack to provide strict type enforcement, facilitating complex data modeling and reducing runtime errors during information formatting.

Which results in the following structure:

```
\background
| index.ts
| nlp.worker.ts
```

```
\lib
| db.ts
\sidepanel
| app.css
| index.html
| main.ts
| SidePanel.svelte
vite.config.ts
```

The project's directory structure reflects a modular separation of concerns across the technical stack. The **development framework** is anchored by `vite.config.ts`, which orchestrates the build process via Vite and CRXJS. The **front end** is contained within the `\sidepanel` directory, utilizing Svelte components (`SidePanel.svelte`) and TypeScript logic (`main.ts`) to render the user interface. Finally, the **back end** logic is distributed between the `\background` directory—which houses the service worker (`index.ts`) and the off-main-thread `nlp.worker.ts` for natural language processing—and the `\lib` directory, which manages local-first data persistence through `db.ts`.

5.2 Development framework

Vite provides a quick and simple framework for me to outline essential extension tools and rapidly test & update Trailhead.

Primarily, Trailhead relies on the following setup:

```
1 background: {
2   service_worker: 'src/background/index.ts',
3   type: 'module',
4 },
5 side_panel: {
6   default_path: 'src/sidepanel/index.html',
7 },
8 permissions: ['sidePanel', 'contextMenus', 'storage', '
  activeTab', 'scripting', 'clipboardWrite']
```

Listing 1: Manifest Configuration for Trailhead

`sidePanel` and `contextMenus` are used to implement the primary user interface, specifically allowing for a persistent research companion and a low-friction method for data entry. The `sidePanel` serves as the central hub for the "mental map" and session management, while the `contextMenus` enable the "select to add snippets" feature, allowing users to right-click highlighted text to instantly "blaze a trail" without leaving their current tab.

Furthermore, the storage and scripting permissions facilitate the bridge between the frontend and the data layer. While storage is used for lightweight configuration and state persistence, the scripting and `activeTab` permissions allow Trailhead to interact with the DOM of the research target to extract metadata, such as DOI numbers for automatic citation fetching, ensuring a seamless flow from raw information to organized knowledge.

5.3 Front End

The Trailhead user interface is architected into two primary functional regions: the **"Session Rail"** and the **"Workspace."** This dual-panel layout adopts the prevailing design language of modern productivity tools—such as Overleaf and contemporary LLM interfaces—where high-level navigation is sequestered to a narrow sidebar, leaving the expansive primary panel for deep focus and interaction.

The Session Rail (Navigation): Located on the left, this panel serves as the management layer for the user’s research history. It allows for the rapid creation, selection, and switching between different "trails" or topics, enabling low-friction context switching.

The Workspace (Interaction): The larger right-hand section serves as the active research environment. It dynamically renders the current session’s snippets, allows for $\text{\LaTeX}$ note-taking, and provides the primary interface for managing the snippets.

The following code snippet for creating a new session provides a simple example of Trailhead’s front end workflow:

```
1 async function createSession() {
2   const title = newSessionTitle.trim();
3
4   if (!title) {
5     return;
6   }
7
8   const now = Date.now();
9   const session: TrailheadSession = {
10    id: crypto.randomUUID(),
11    title,
12    createdAt: now,
13    updatedAt: now,
14  };
15
16  const db = await dbPromise;
17  await db.add('sessions', session);
18  newSessionTitle = '';
19  sessionFormOpen = false;
20  await loadSessions();
21  await setActiveSession(session.id);
22 }
```

Listing 2: Citation Export and Clipboard Integration

The `createSession` function illustrates the asynchronous nature of Trailhead’s architecture, utilizing `async/await` to ensure that heavy database operations do not block the user interface. By interfacing with IndexedDB via the `db.add` method, the tool ensures that every research session is persisted locally and assigned a unique `crypto.randomUUID()` for reliable indexing. This approach facilitates a "local-first" data model, where session metadata and discovery trails are

stored securely within the browser, ensuring both privacy and low-latency retrieval.

Similarly, the `async` function `createEntry` compiles and adds to the current database the following:

```
1 await db.add('entries', {
2   id: crypto.randomUUID(),
3   sessionId: activeSessionId,
4   url: newEntry.url.trim(),
5   text,
6   note,
7   kind: isScreenshot ? 'screenshot' : 'text',
8   screenshotDataUrl,
9   screenshotMimeType,
10  citation,
11  needsCitation: false,
12  keywords: [],
13  timestamp: now,
14  updatedAt: now,
15 });
```

Listing 3: Citation Export and Clipboard Integration

Beyond session management, the front end serves as the primary data orchestrator, fetching stored information from the IndexedDB backend and transforming it into user-facing outputs. This is most evident in the citation and export workflow.

The functions `exportCitation` and `exportSessionCitations` demonstrate how Trailhead connects the internal database and the user’s external research document:

```
1 async function exportCitation(entry: TrailheadEntry) {
2   if (!entry.citation) {
3     return;
4   }
5   await navigator.clipboard.writeText(entry.citation);
6   exportedCitationEntryId = entry.id;
7   openSnippetMenuId = '';
8 }
9 async function exportSessionCitations() {
10  if (!uniqueSessionCitations.length) {
11    return;
12  }
13  await navigator.clipboard.writeText(
14    uniqueSessionCitations.join('\n\n'));
15  exportedSessionCitations = true;
16 }
```

Listing 4: Citation Export and Clipboard Integration

The "parse-on-insertion" model ensures that processing is done asynchronously, preventing UI "jank" during intensive research sessions. By offloading keyword extraction to the `nlp.worker.ts`, Trailhead prepares the data for near-instantaneous retrieval. This shifts the computational load to the moment of capture, allowing the associative search index

to remain updated in real-time without requiring expensive full-text scans during user queries.

A lot of less critical code is dedicated to features such as hidden buttons and state-aware cues (e.g. asking for notes when user tries to store a screenshot).

5.4 Back End

The Trailhead backend is designed as a non-persistent, event-driven service layer that operates independently of the UI. It is primarily responsible for global event listening, high-latency data processing, and managing the integrity of the local-first database.

The core of the backend is the background/index.ts service worker. Unlike the sidepanel, which can be closed by the user, the service worker remains active in the background to handle system-level triggers. For example, it manages the **Context Menu API**, which allows users to capture snippets from any tab, and when a user selects text and chooses "Add to Trailhead," the background script intercepts the event, captures the active tab's metadata (URL, Title, and DOI), and dispatches the data to the storage layer.

```
1 chrome.contextMenus.onClicked.addListener(async (info,
2   tab) => {
3   if (info.menuItemId === 'save-to-trailhead' && tab?.url)
4     {
5     openSidePanelFromGesture(tab);
6
7     const text = info.selectionText || "Full page captured
8       ";
9
10    if (typeof Worker === 'undefined') {
11      await saveEntry(tab, text);
12      return;
13    }
14  }
```

Listing 5: Sample listener

Data persistence is managed via the lib/db.ts module, which abstracts **IndexedDB** using the idb wrapper. By utilizing a "transactional" approach, the backend ensures that data insertion remains atomic and thread-safe. As noted in the "parse-on-insertion" model, the backend does not just store raw strings; it prepares them for retrieval by indexing them into separate object stores for sessions and entries.

```
1 export interface TrailheadSession {
2   id: string;
3   title: string;
4   createdAt: number;
5   updatedAt: number;
6 }
7
8 export interface TrailheadEntry {
9   id: string;
10  sessionId: string;
```

```
11 url: string;
12 text: string;
13 note: string;
14 kind: 'text' | 'screenshot';
15 screenshotDataUrl: string;
16 screenshotMimeType: string;
17 citation: string;
18 needsCitation: boolean;
19 keywords: string[];
20 timestamp: number;
21 updatedAt: number;
22 }
23
24 export interface TrailheadDB extends DBSchema {
25   sessions: {
26     key: string;
27     value: TrailheadSession;
28   };
29   entries: {
30     key: string;
31     value: TrailheadEntry;
32     indexes: { 'by-session': string };
33   };
34 }
35 }
```

Listing 6: DB structure

To maintain a responsive experience, computationally expensive tasks—such as keyword extraction and associative indexing—are delegated to the nlp.worker.ts.

```
1 // Inside nlp.worker.ts
2 import nlp from 'compromise';
3
4 self.onmessage = async (e) => {
5   const { text, id } = e.data;
6
7   // Rule-based entity extraction
8   const doc = nlp(text);
9   const keywords = doc.nouns().out('array').slice(0, 5);
10
11   // Update the associative index without blocking the UI
12   await updateSearchIndex(id, keywords);
13
14   self.postMessage({ id, keywords, status: 'complete' });
15   };
```

Listing 7: Background Keyword Extraction and Indexing

This worker-based architecture allows the backend to perform "heavy lifting" (like running the Compromise.js NLP engine) without introducing latency to the SidePanel's animations or user interactions. By the time a user opens their search interface, the backend has already asynchronously generated the tags and associative links required for a "mental map" view.

6 Usage

The following section will share the most primary usages, alongside screenshots.

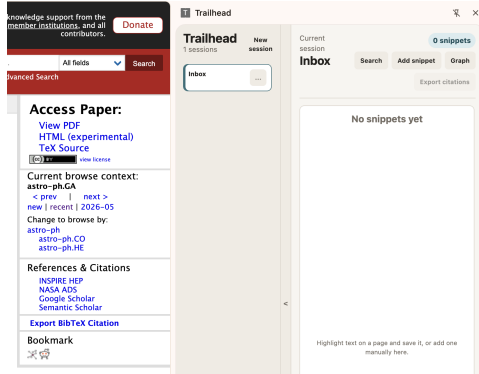


Figure 1: Default View of Trailhead

Fig. 1 shows the default view of Trailhead, when opened from the extension button on top-right of browser. Individual actions for sessions and entries (not shown in this figure) are hidden under the "... " button for clarity.

6.1 select to add snippets

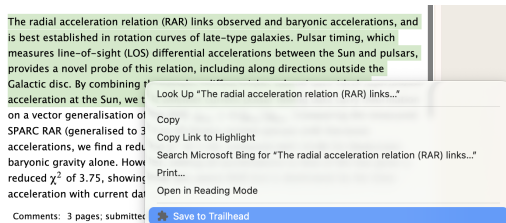


Figure 2: Sample Usage for adding snippets

With or without the UI open, users with Trailhead installed can select a chunk of text to add as a snippet to the current session.

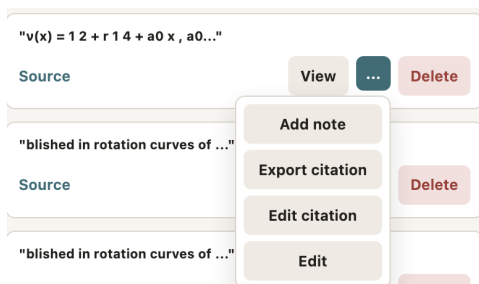


Figure 3: Snippet Interaction

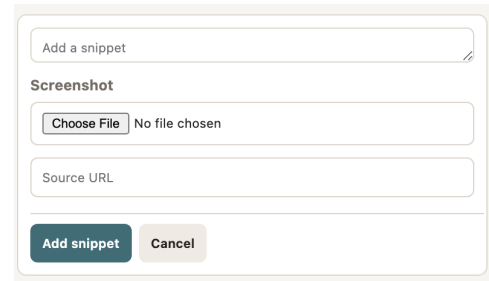


Figure 4: Add_Snippet Feature

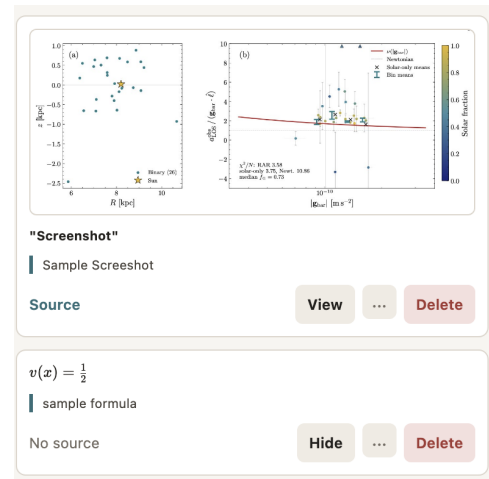


Figure 5: Sample Usage for Screenshot and LATEXFormula

6.2 screenshots

Fig 4 demonstrates the standard usage to add custom snippets. The "Add snippet" Button on the top of the work space toggles a window to manually enter fields for a snippet. Users can drag or paste a screenshot or a latex expression, as well as notes and their source. For searching purpose, note is mandatory for screenshots.

Fig 5 Shows how the two will be represented. The smaller text that begin with a green bar is the custom note added to the snippet. Note that the button on the formula block shows "Hide": The view feature will show full text for snippets and render all latex expressions, while the default view keep only the first 30 chars to save space. Images are unaffected by the view feature.

6.3 Automatic citation & one click export

The worker automatically looks for citations on the page. If a proper doi was not found, a note will be shown like Fig. 6, prompting the user to add citations manually via the feature menu shown in Fig. 3.

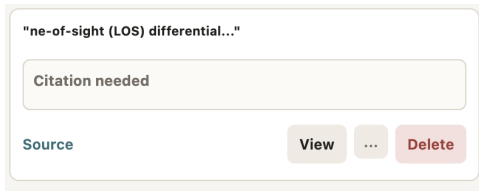


Figure 6: Snippet without citation

Fig 7 shows how one would export all the citations from the current session, using the "Export citations" (Now showing "Copied citations") button on the menu. As seen in previous code examples, the citations have already been deduplicated so that two snippets from the same source will have only one citation stored in the global list. One could also use the export button in Fig 3 to copy a single citation.

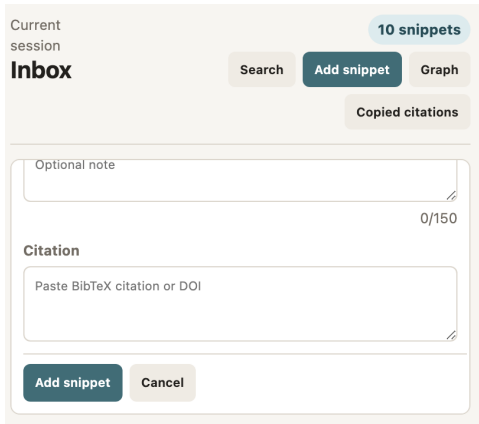


Figure 7: Sample citation export

6.4 search

Similar to exporting citations and adding snippets, the search function is a toggled window through the "Search" button on the top. This will perform a text-based search within the session, effectively providing a human-distilled search index. Notes will have a higher relevance score than the texts in the snippets, as user's input is deemed to be the best indicator of the topic.

6.5 graph view

Fig 8 provides an example of the graph view. Each node represents a snippet, while each edge suggests a relationship based on cosine similarity. A stronger relation is represented with a thicker connection.

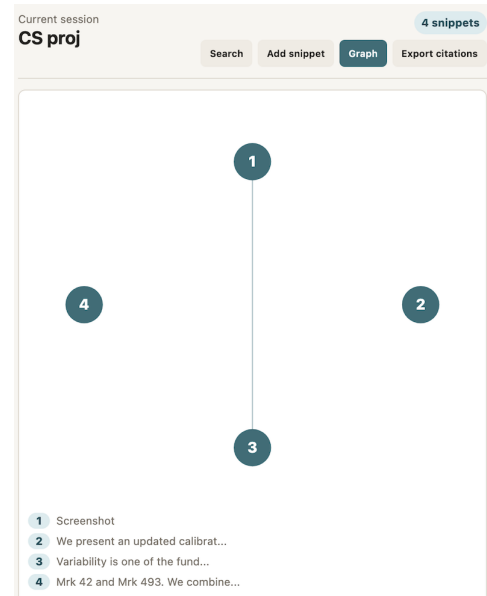


Figure 8: Sample graph

7 Evaluation

Evaluation is done primarily by usability testing. I recruited two users, both are students in the STEM field who uses $\text{\LaTeX}$ and frequently look for information in arXiv for research. The evaluation is based on my own usage, and a simple usability test with the recruited students. Before using it, a simple description of the purpose and features of the tool is given to each participant. A small discussion focusing on use experience and disclosure of current limitations is conducted after the 10 min use test.

7.1 Evaluation Findings

While both participants find the base usage to be helpful, there were indeed several issues with the current version of Trailhead.

- (1) **Graph View:** currently, the graph view shows a ring of nodes and a list of each node's corresponding snippet below. This is readable only with small snippet counts. The current version of graph does not support focusing on a specific node, making the graph view somewhat useless.
- (2) **Custom Input:** the custom snippet input remains to be a labor-intensive action, requiring the user to manually fill in all fields. Some of the fields, for instance the source url and citation could potentially add that of the current view page by default.
- (3) **Implemented feedback:** Features such as in-line latex rendering and paste image from clipboard are

results from the earlier rounds of feedback. They have been implemented.